CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

Project On

SMART BANKING SYSTEM

SUBMITTED TO:

MR JAGVINDER SIR

SUBMITTED BY:

MUKUL SHARMA

UNIVERSITY SCHOOL OF BUSINESS

CHANDIGARH UNIVERSITY

GHARUAN, MOHALI (PUNJAB)

CERTIFICATE

This is to certify that the project entitled “Smart Banking System Using C++” has been developed and submitted by the Mukul Sharma as part of the academic requirements for the Bachelor of Computer Applications (BCA).

The project demonstrates the practical implementation of Object-Oriented Programming concepts, including classes, objects, encapsulation, inheritance, abstraction, polymorphism, exception handling, and file handling.

The work has been completed under the guidance and supervision of the concerned faculty member.

Signature:

Date:

Department: BCA

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty members and department for providing me with the opportunity to develop the Smart Banking System project.

I am thankful to my teacher for the guidance and support provided throughout the development of this project. This project has helped me understand the practical applications of Object-Oriented Programming in C++.

I would also like to thank my classmates, friends, and everyone who supported me during the planning, coding, testing, and documentation of this project.

Finally, I am grateful for the opportunity to enhance my programming skills and gain practical knowledge of software development through this project.

INDEX

1. Introduction
2. Problem Statement
3. Objectives of the Project
4. Scope of the Project
5. Object-Oriented Programming Concepts
6. System Modules
7. Implementation and Source Code
8. Sample Output
9. Software Testing
10. Advantages and Limitations
11. Future Enhancements
12. Conclusion
13. References

INTRODUCTION

Banking systems play an important role in managing financial transactions, maintaining customer accounts, and recording deposits and withdrawals. Traditionally, many banking activities required customers to visit a bank branch and interact with bank employees.

With the advancement of computer technology, banking operations can be managed through software applications that improve efficiency, accuracy, and record management.

The Smart Banking System is a C++ application that demonstrates the basic working of a banking system. It allows users to perform common banking activities through a menu-driven interface.

The system follows Object-Oriented Programming principles to organize the program into reusable classes and objects. It also uses file handling to store information permanently between program executions.

Object-Oriented Programming is a programming approach that organizes software around objects containing data and functions.

C++ supports OOPS concepts such as classes, objects, encapsulation, inheritance, abstraction, and polymorphism. These concepts are useful for developing applications that contain multiple entities and operations.

A banking application is a suitable example because it contains entities such as bank accounts, customers, transactions, and different account types.

The purpose of the project is to demonstrate how C++ can be used to create a structured application that simulates banking operations.

The project also aims to improve understanding of class design, data validation, file storage, transaction processing, and error handling.

PROBLEM STATEMENT

A basic banking record system must maintain account information and allow users to perform financial operations accurately.

A poorly designed system may face problems such as duplicate account numbers, incorrect balance updates, invalid withdrawals, loss of transaction records, and difficulty retrieving account information.

The Smart Banking System addresses these problems within a small educational application.

- Maintaining account records manually can be time-consuming.
- Calculating balances manually can lead to errors.
- Searching for customer information may take additional time.
- Transaction records may be difficult to maintain consistently.
- Account information may be lost if records are not preserved properly.

The proposed system provides a centralized program for managing basic banking operations.

It validates transaction amounts, prevents withdrawals exceeding the available balance, verifies account numbers, and stores account records in files.

The application demonstrates how software can make record management more organized and repeatable.

OBJECTIVES OF THE PROJECT

The main objectives are:

1. To develop a banking simulation using C++.
2. To implement classes and objects for banking entities.
3. To apply encapsulation to protect account balances.
4. To use inheritance for different types of bank accounts.
5. To implement abstraction and runtime polymorphism.
6. To support account creation and account information display.
7. To implement deposit, withdrawal, and fund transfer operations.
8. To maintain a transaction history.
9. To use file handling for persistent storage.
10. To validate user inputs and handle exceptional conditions.
11. To improve programming, debugging, and software testing skills.

SCOPE OF THE PROJECT

The system supports the following functions:

- Creation of bank accounts.
- Assignment of unique account numbers.
- Display of account details.
- Deposit of money.
- Withdrawal of money.
- Transfer of funds between existing accounts.
- Balance inquiry.
- Transaction history viewing.

- Different account types with different interest-rate policies.
- Saving and loading account records through files.

The project is suitable for demonstrating C++ programming and OOPS concepts in an academic environment.

It can be extended with a graphical interface, database connectivity, authentication, and additional banking features.

The system does not connect to actual banks, payment networks, or financial institutions. It does not provide real online banking, identity verification, regulatory compliance, or production-grade financial security.

OBJECT-ORIENTED PROGRAMMING CONCEPTS

A class is a blueprint that defines data members and member functions.

An object is an instance of a class.

In this project, Bank Account defines the common structure of an account, while individual account objects represent specific customer accounts.

Encapsulation combines data and related functions into one unit and controls access to the internal data.

The account balance is kept private. Users cannot directly change it; they must use controlled operations such as deposit and withdrawal.

This helps maintain the validity of account balances.

Inheritance allows one class to acquire features from another class.

Savings Account and Current Account inherit common functionality from Bank Account.

This reduces duplication and makes the program easier to extend.

Abstraction exposes essential functionality while hiding implementation details.

The abstract Bank Account class defines common banking operations without requiring every account type to implement the same internal logic.

Polymorphism allows the same function interface to produce different behaviour for different object types.

For example, the virtual interest Rate() function returns different rates for savings and current accounts.

When called through a base-class pointer, the correct derived-class implementation is selected at runtime.

Constructors initialize objects when they are created.

In this project, constructors assign account numbers, customer names, and opening balances.

Exception handling allows the program to respond to invalid operations without relying entirely on normal execution.

For example, invalid transaction amounts and withdrawals exceeding the available balance can be rejected.

File handling allows information to be stored in files and retrieved later.

The project uses file streams to read and write account records and transaction logs.

Virtual functions support runtime polymorphism.

The project uses virtual functions for account-type identification and interest-rate calculation.

The application separates tasks into individual functions, such as account creation, balance inquiry, deposits, withdrawals, and transfers.

This improves readability, testing, and maintenance.

SYSTEM MODULES

This module collects the customer’s name, account type, and opening balance.

It assigns a unique account number and saves the new account.

The module accepts an account number and deposit amount. It verifies the account, validates the amount, and updates the balance.

The module verifies the account and checks the amount requested.

If sufficient funds are available, the balance is reduced. Otherwise, an error message is displayed.

The transfer module accepts the sender's account number, recipient's account number, and transfer amount.

It checks whether both accounts exist and whether the sender has sufficient funds.

The sender's balance is reduced and the recipient's balance is increased. Both changes should be treated as one logical operation.

This module retrieves and displays the current balance of a selected account.

This module displays the available account records, including account number, customer name, account type, and balance.

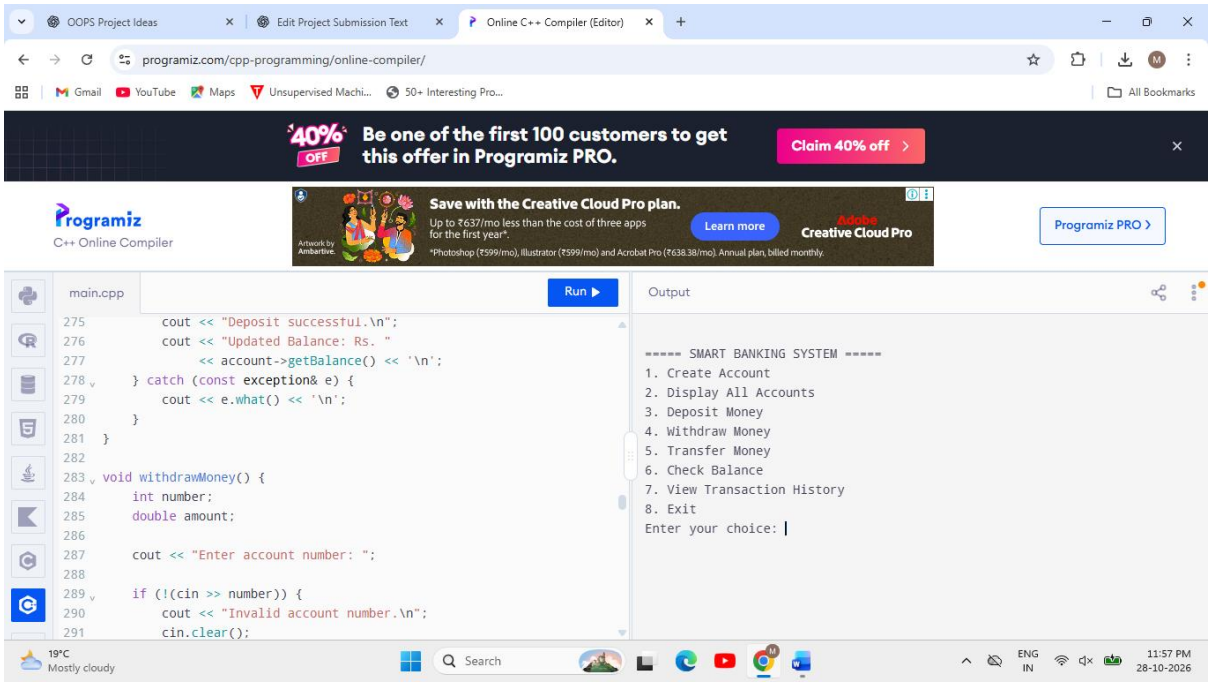
This module displays previously recorded transactions from the transaction file.

This module loads stored accounts at startup and writes updated records to disk when necessary.

IMPLEMENTATION AND SOURCE CODE

The application is implemented in C++ using classes, inheritance, virtual functions, vectors, and file streams.

The following source code is an educational implementation of the banking system. It supports basic account management, deposits, withdrawals, transfers, and persistent account records.



Run it on Windows with:

banking.exe

The program creates accounts.txt and transactions.txt in the working directory when account data or transactions are saved.

Implementation note: This is a teaching prototype, not a secure financial application. Production use would require atomic transaction handling, reliable recovery after write failures, stronger input and data validation, authentication, and secure storage.

SAMPLE OUTPUT

The following examples illustrate the expected interaction. They are sample outputs, not screenshots from an executed program.

===== SMART BANKING SYSTEM =====

1. Create Account

2. Display All Accounts

3. Deposit Money

4. Withdraw Money

5. Transfer Money

6. Check Balance

7. View Transaction History

8. Exit

Enter customer name: Rahul Sharma

1. Savings Account

2. Current Account

Select account type: 1

Enter opening balance: 5000

Account created successfully.

Account Number: 1001

Enter account number: 1001

Enter deposit amount: 1000

Deposit successful.

Updated Balance: Rs. 6000.00

Enter account number: 1001

Enter withdrawal amount: 500

Withdrawal successful.

Remaining Balance: Rs. 5500.00

SOFTWARE TESTING

Testing checks whether the system behaves as expected for valid inputs, invalid inputs, and different transaction conditions.

The following are planned test cases. They should be executed against the final compiled program before submission.

ADVANTAGES AND LIMITATIONS

Advantages

1. Simple and understandable menu-driven interface.
2. Organized code using classes and objects.
3. Demonstration of major OOPS principles.
4. Support for multiple account types.
5. Basic validation of deposits and withdrawals.
6. Fund transfer between accounts.
7. Persistent storage through local files.
8. Transaction logging.
9. Modular design that can be extended later.
10. Low development cost and minimal hardware requirements.

Limitations

1. The program does not use a real banking database.
2. It does not provide a graphical user interface.
3. It does not implement customer authentication.
4. Text files are not protected by encryption.
5. The transaction log is basic and does not provide complete audit guarantees.
6. File-write failures can create inconsistencies.
7. The program is not designed for simultaneous users.
8. It does not support online payments or real bank connectivity.
9. Interest rates are demonstration values rather than verified banking products.
10. It is not suitable for handling real customer funds.

FUTURE ENHANCEMENTS

The Smart Banking System can be enhanced in several ways.

A graphical interface can be developed using Qt or another suitable framework to improve usability.

MySQL or SQLite can replace text files to support structured records, constraints, and more reliable data management.

A login system with secure password hashing and access controls can be added.

Transfers can be implemented using database transactions or an equivalent atomic storage mechanism to prevent partial updates.

The system can be extended to manage loan applications, repayments, and interest calculations.

A separate module can calculate maturity amounts and interest according to configurable rules.

The project can include PIN verification, withdrawal limits, and cash-availability simulation.

The system can generate account statements, transaction summaries, and printable reports.

An administrative interface can manage account status, account categories, and system-level settings.

A real financial application would require professional security engineering, audit trails, access controls, backup and recovery mechanisms, and compliance with applicable regulations.

CONCLUSION

The Smart Banking System successfully demonstrates the application of C++ programming and Object-Oriented Programming concepts to a practical problem.

The project organizes banking data and operations using classes, inheritance, abstraction, encapsulation, and polymorphism. It also introduces file handling for persistent records and exception handling for invalid operations.

Through account creation, deposits, withdrawals, balance inquiry, money transfers, and transaction history, the project demonstrates how a structured software application can simulate basic banking activities.

The development process provides valuable practical experience in program design, modular coding, input validation, testing, and documentation.

Although the system is intended only for educational use, its design provides a foundation for future improvements involving databases, graphical interfaces, stronger security, and more advanced transaction management.

In conclusion, the project meets its educational objective of demonstrating the practical use of OOPS concepts in C++.

REFERENCES

The following resources can be consulted while studying and developing the project.

1. Bjarne Stroustrup, *Programming: Principles and Practice Using C++*.
2. C++ language documentation: https://en.cppreference.com/w/
3. GCC documentation: https://gcc.gnu.org/onlinedocs/
4. Microsoft C++ documentation: https://learn.microsoft.com/en-us/cpp/
5. Course notes and learning materials provided by the university.
6. C++ standard-library documentation covering classes, exceptions, vectors, and file streams.